

E-Ponto

Sistema Web de Registro Eletronico de Ponto (REP-P)

Documentacao Tecnica do Projeto

Conformidade com a Portaria MTP n. 671/2021

Disciplina: Programacao Avancada para Web

Instituicao: Universidade Vila Velha (UVV) - 5o periodo

Produto: E-Ponto

Grupo: Grupo 5

Integrantes do grupo:

- Roberto Dias Furtado de Araujo - 202421518
- Rychard Chaves Brandao - 202417506
- Lucas Andre - 202422326

Stack principal: Python 3.12 + Flask (Application Factory) + SQLAlchemy 2.x + Flask-Migrate + Flask-Login + Flask-WTF + Bootstrap 5 / Jinja2.

1. Descricao geral do produto

O **E-Ponto** e uma plataforma web para registro eletronico de ponto, voltada a empresas de pequeno e medio porte (ate ~200 funcionarios). Diferente de um cadastro comum, ele implementa as exigencias legais brasileiras de um **REP-P** (Registro Eletronico de Ponto via Programa), descritas na Portaria MTP n. 671/2021: Numero Sequencial de Registro (NSR), cadeia de hashes para inviolabilidade e geracao dos arquivos oficiais AFD e AEJ. O sistema e **multi-empresa** (multi-tenant): um mesmo usuario pode ter papeis diferentes em empresas diferentes.

Funcionalidades principais

- Autenticacao por e-mail e senha (hash Bcrypt) com **2FA opcional** (TOTP via pyotp + QR Code).
- Controle de acesso por papeis (RBAC): super_admin, admin, rh, funcionario - com escopo por empresa.
- Registro de ponto com 4 batidas (Entrada, Saida Almoco, Retorno Almoco, Saida Final), data/hora em UTC e **geolocalizacao**.
- **Geofencing**: cada batida e comparada ao raio do local de trabalho; fora do raio fica marcada como suspeita para revisao do RH.
- Comprovante de cada batida em tela e em **PDF** (ReportLab).
- Dashboard do funcionario: saldo de horas, banco de horas, atrasos, faltas e calendario do mes.
- Gestao de RH: listagem/filtros de registros, aprovacao de **retificacoes** (sem sobrescrever o historico) e relatorios.
- Exportacao de relatorios em **Excel (.xlsx)** (Pandas + openpyxl) e geracao dos arquivos legais **AFD** e **AEJ**.
- Trilha de **auditoria** (audit_logs) das acoes sensiveis e rate limiting nas rotas de login.

2. Diagrama do banco de dados (modelo ER)

O schema possui **15 tabelas** (modeladas com SQLAlchemy 2.x e versionadas via Alembic/Flask-Migrate). O diagrama textual abaixo resume os relacionamentos centrais; a tabela seguinte lista todas as entidades.

```
users 1---* roles_has_users *---1 roles (RBAC: papel por empresa)
      | * business_id, level_id (opc.)
      v 1
      businesses 1---* jornadas
            | 1 1---* locais_trabalho
            | *
            escalas_funcionarios (vincula user + jornada + local)
            |
            v
businesses 1---* registros *---1 users (a batida de ponto)
          | | *---? locais_trabalho
          | +---* retificacoes *---1 users (solicitante/aprovador)
          |
          +---1 nsr_sequencias (contador NSR da empresa)
          +---* banco_horas *---1 users (saldo mensal)
          +---* audit_logs (trilha de auditoria)

users 1---* address *---? cities (endereço do funcionario)
```

Legenda: 1---* (um para muitos), *---1 (muitos para um), ? (opcional / FK anulavel).

Tabela	Descricao	Chaves / FKs principais
users	Usuario (CPF, PIS/NIS, senha bcrypt, totp_secret, cargo, data_admissao)	PK id; data_admissao
roles	Papeis: super_admin, admin, rh, funcionario	PK id; name unico
levels	Niveis organizacionais opcionais	PK id
roles_has_users	Associativa RBAC (user x role x empresa)	FK user_id, role_id, business_id, level_id; UQ(user,role,business_id,level_id)
businesses	Empresa empregadora (CNPJ, CEI/CAEPF, endereco, cidade)	PK id; FK owner_user_id; cnpj unico
address / cities	Endereco do funcionario e cidades	FK user_id, city_id
locais_trabalho	Locais fisicos com lat/lon + raio (geofence)	PK id; FK empresa_id
jornadas	Jornada de trabalho (padrao, 12x36, 6x1, flexivel)	PK id; FK empresa_id
escalas_funcionarios	Vinculo funcionario-jornada-local com datas	FK user_id, empresa_id, jornada_id, local_trabalho_id
registros	Batida de ponto (nucleo do sistema)	PK id; FK empresa_id, user_id, local_trabalho_id; UQ(empresa_id,user_id,local_trabalho_id,data)
nsr_sequencias	Contador sequencial de NSR por empresa	PK empresa_id
retificacoes	Pedido de correcao (PENDENTE/APROVADA/REJEITADA)	FK registro_id, empresa_id, solicitante_id, aprovador_id
banco_horas	Saldo mensal (positivos/negativos/saldo)	FK user_id, empresa_id; UQ(user,empresa,periodo)
audit_logs	Trilha de auditoria (dados_antes/dados_depois)	FK empresa_id, user_id

3. Arquitetura adotada

A aplicacao segue o padrao **Application Factory** do Flask: nao existe um objeto global de app, e sim a funcao **create_app()** que monta a instancia sob demanda. Isso evita imports circulares e permite criar uma instancia de teste isolada (banco em memoria) separada da de producao. Cada extensao expoe a mesma convencao **init_app(app)**.

```
def create_app(test_config=None):
    app = Flask(__name__)
    init_config(app) # 1) le .env -> app.config
    if test_config:
        app.config.update(test_config)
    init_db(app); register_models() # 2) SQLAlchemy + carrega 15 models
    init_auth(app) # 3) Flask-Login
    init_migrate(app) # 4) Flask-Migrate / Alembic
    init_mail(app) # 4.5) Flask-Mail
    if not app.config.get("TESTING"):
        init_wtf(app) # 5) CSRF (off em testes)
        init_limiter(app) # Rate limiting (off em testes)
    init_toolbar(app) # 6) Debug Toolbar (dev)
    init_site(app) # 7) Registra os 6 blueprints
    return app
```

app.py - funcao create_app() (resumida)

A organizacao em camadas separa responsabilidades de forma clara:

```
E_Ponto/
app.py # factory create_app()
tasks.py # comandos Invoke (run/test/seed/migrate/zip)
pyproject.toml # dependencias (PEP 621)
.env.{dev,test,prod}
E_Ponto/
  ext/ # config, db, auth, wtf, limiter, migrate, mail, debugtoolbar
  models/ # 15 modelos SQLAlchemy
  views/ # 6 blueprints (auth, main, ponto, admin, rh, funcionario)
  forms/ # WTForms (validacao no servidor)
  utils/ # nsr, hashing, geo, clt, afd, aej, pdf, excel, decorators
  templates/ # Jinja2 + Bootstrap 5
  tests/ # 122 testes (pytest)
  migrations/ # versoes Alembic
```

A camada de rotas e dividida em **6 Blueprints**, um por dominio: **auth** (/auth - login, 2FA), **main** (/ - home e selecao de empresa), **ponto** (/ponto - bater, historico, comprovante), **funcionario** (/funcionario - dashboard), **rh** (/rh - registros, retificacoes, relatorios) e **admin** (/admin - usuarios, jornadas, locais). As regras de negocio ficam isoladas em utils/, mantendo as views finas e testaveis.

4. Detalhamento tecnico das principais partes

4.1 Modelagem (SQLAlchemy 2.x)

Os modelos usam o estilo moderno Mapped/mapped_column. A tabela **registros** e a mais rica: usa Enum (evita typos), guarda o timestamp sempre em UTC, geolocalizacao e os campos da cadeia de hash. Repare na UniqueConstraint que garante NSR unico por empresa.

```
class TipoRegistro(enum.Enum):
    ENTRADA = "entrada"; SAIDA_ALMOCO = "saida_almoco"
    RETORNO_ALMOCO = "retorno_almoco"; SAIDA = "saida"
    INCLUSAO = "inclusao"; ALTERACAO = "alteracao"

class Registro(db.Model):
    __tablename__ = "registros"
    __table_args__ = (
        UniqueConstraint("empresa_id", "nsr", name="uq_empresa_nsr"),
        {'extend_existing': True})
    id: Mapped[int] = mapped_column(db.Integer, primary_key=True)
    nsr: Mapped[int] = mapped_column(db.Integer, nullable=False)
    tipo: Mapped[TipoRegistro] = mapped_column(db.Enum(TipoRegistro))
    timestamp_utc: Mapped[datetime] = mapped_column(
        db.DateTime(timezone=True), nullable=False, index=True)
    latitude: Mapped[Optional[float]] = mapped_column(Numeric(10, 7))
    suspeito_geo: Mapped[bool] = mapped_column(db.Boolean, default=False)
    hash_registro: Mapped[str] = mapped_column(db.String(64), nullable=False)
    hash_anterior: Mapped[Optional[str]] = mapped_column(db.String(64))
```

E_Ponto/models/registro.py (resumido)

4.2 Formularios e validacao (Flask-WTF)

Toda entrada do usuario passa por validacao **no servidor**. O formulario de bater ponto recebe a geolocalizacao via campos ocultos (preenchidos por JavaScript) e a view so executa a logica apos form.validate_on_submit().

```
class BaterPontoForm(FlaskForm):
    tipo = SelectField('Tipo', choices=[('entrada', 'Entrada'), ...])
    latitude = HiddenField('Latitude') # preenchido por JS (Geolocation)
    longitude = HiddenField('Longitude')
    local_trabalho_id = SelectField('Local', coerce=int,
                                    validators=[Optional()])
    justificativa = TextAreaField('Justificativa',
                                   validators=[Optional(), Length(max=255)])
    submit = SubmitField('Registrar Ponto')
```

E_Ponto/forms/ponto.py

4.3 Integridade: NSR atomico + cadeia de hash

Ao bater ponto, a view gera o proximo NSR de forma atomica (lock de linha) e calcula um SHA-256 encadeado ao hash do registro anterior. Assim, alterar uma batida antiga invalidaria todos os hashes seguintes - a exigencia anti-adulteracao do REP-P.

```
def get_next_nsr(empresa_id: int) -> int:
    seq = (NsrSequencia.query.filter_by(empresa_id=empresa_id)
          .with_for_update().first()) # SELECT ... FOR UPDATE
    if seq is None:
        seq = NsrSequencia(empresa_id=empresa_id, ultimo_nsr=0)
        db.session.add(seq)
    seq.ultimo_nsr += 1
    db.session.flush()
    return seq.ultimo_nsr
```

```
def calcular_hash(nsr, cnpj, pis_cpf, timestamp_iso, tipo, hash_anterior):
    data = f"{nsr}|{cnpj}|{pis_cpf}|{timestamp_iso}|{tipo}" \
           f"{hash_anterior or 'GENESIS'}"
    return hashlib.sha256(data.encode('utf-8')).hexdigest()
```

E_Ponto/utils/nsr.py e utils/hashing.py

4.4 Geofencing (formula de Haversine)

```
def verificar_geofence(lat, lon, local_trabalho) -> tuple:
    if lat is None or lon is None or local_trabalho is None:
        return True, 0.0 # sem dados: nao da pra julgar
    dist = haversine(lat, lon, local_trabalho.latitude,
                     local_trabalho.longitude)
    return dist <= local_trabalho.raio_metros, dist
```

E_Ponto/utils/geo.py

4.5 Testes automatizados (pytest)

Sao **122 testes** (76 funcoes, varias parametrizadas) cobrindo rotas, calculos, seguranca/autorizacao e regras de negocio. A fixture principal sobe a app em modo teste com **SQLite em memoria** isolado por teste, e o TZ e fixado em UTC para deixar os calculos de hora determinísticos no CI.

```
@pytest.fixture
def app():
    app = create_app(test_config={
        "TESTING": True, "SECRET_KEY": "test-secret-key",
        "SQLALCHEMY_DATABASE_URI": "sqlite://", # banco em memoria
        "SQLALCHEMY_ENGINE_OPTIONS": {"poolclass": StaticPool, ...},
        "WTF_CSRF_ENABLED": False})
    ctx = app.app_context(); ctx.push()
    _db.create_all(); ctx.pop()
    yield app
    ... # drop_all no teardown
```

Arquivo	Foco	Qtde
test_acesso.py	Controle de acesso por papel (parametrizado)	53
test_unitarios.py	Funcoes puras (calculos, hash)	19
test_invasao.py	Tentativas de invasao / autorizacao	11
test_ponto / test_calculos	Fluxo de ponto e calculos de horas	8
test_auth.py	Login e 2FA	6
demais (smoke, dashboard, rh, ...)	Relatorios, retificacao, erros, etc.	25

5. Tecnologias utilizadas

Tecnologia	Para que serve
Flask	Microframework web; base da Application Factory e dos blueprints
Flask-SQLAlchemy / SQLAlchemy 2.x	ORM e modelagem das 15 entidades
Flask-Migrate / Alembic	Versionamento e migracao do schema
Flask-Login	Sessao de usuario, @login_required, current_user
Flask-WTF / WTForms	Formularios + validacao no servidor + CSRF
Flask-Bcrypt	Hash seguro de senhas
pyotp + qrcode	Autenticacao em 2 fatores (TOTP)
Flask-Limiter	Rate limiting nas rotas sensiveis (login)
Flask-Mail	Notificacao por e-mail (ex.: retificacao decidida)
ReportLab	Geracao de PDF (comprovante e esta documentacao)
Pandas + openpyxl	Exportacao de relatorios em Excel (.xlsx)
Bootstrap 5 + Jinja2	Interface responsiva e templates
SQLite / PostgreSQL	Banco em dev (SQLite) e producao (Postgres)
pytest (+pytest-cov)	Suite de testes automatizados
Invoke	Automacao de comandos (inv run/test/seed/...)
black + flake8	Formatacao e lint

Justificativa: o projeto usa **Flask** por ser o stack visto em aula e por casar bem com o padrao Application Factory + Blueprints. As bibliotecas do ecossistema (SQLAlchemy, WTF, Login, Migrate) sao maduras e integram-se nativamente, o que reduziu o codigo de infraestrutura e deixou o foco nas regras do REP-P. SQLite acelera o desenvolvimento e os testes; PostgreSQL entra em producao apenas trocando a variavel DATABASE_URL.

6. Como rodar o projeto (passo a passo)

```
# 1) Clonar e entrar na pasta
git clone https://github.com/underthedarkxx/E_Ponto.git
cd E_Ponto

# 2) Criar e ativar o ambiente virtual
python3 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# 3) Instalar dependencias (modo editavel + dev/test)
inv install # ou: pip install -e ".[dev,test]"

# 4) Configurar variaveis de ambiente
cp .env.example .env.dev
cp .env.example .env.test
python -c "import secrets; print(secrets.token_urlsafe(48))" # SECRET_KEY

# 5) Criar o banco e popular dados iniciais
inv initdb # cria as tabelas
inv seed # papeis + super_admin + funcionario demo

# 6) Rodar o servidor de desenvolvimento
```

```
inv run # http://127.0.0.1:5000
```

```
# 7) Rodar os testes
```

```
inv test
```

Usuarios criados pelo **inv seed**: admin@eponto.com / admin123 (super_admin) e joao@eponto.com / joao123 (funcionario). Demais comandos uteis: **inv lint** (flake8), **inv format** (black), **inv migrate -m "..."** / **inv upgrade** (migracoes) e **inv zip** (empacota o projeto sem venv/cache).

7. Principais desafios e como foram resolvidos

Inviolabilidade dos registros (exigencia legal)

O REP-P exige que registros não possam ser adulterados. Resolvemos com uma cadeia de hashes SHA-256: cada batida inclui o hash da anterior, então alterar uma antiga invalida todas as seguintes. Um utilitário (`verificar_integridade`) percorre a cadeia e aponta onde quebrou.

NSR unico sob concorrência

Duas batidas simultâneas poderiam receber o mesmo NSR. Usamos `with_for_update()` (`SELECT ... FOR UPDATE`) para travar a linha do contador até o commit, garantindo números sequenciais sem colisão em Postgres; o controle é centralizado em `get_next_nsr()`.

Fuso horário e horário de verão

Misturar fusos causava erros de cálculo de atraso/horas. Padronizamos gravar tudo em UTC (`timestamp_utc`) e converter para o horário local apenas na exibição, via filtros Jinja (`localdt`). Nos testes, o TZ é fixado em UTC para resultados determinísticos.

Validar localização do funcionario (geofence)

Precisávamos saber se a batida ocorreu no local certo. Implementamos a fórmula de Haversine para medir a distância entre o GPS do navegador e o local cadastrado; fora do raio, a batida é aceita mas marcada como suspeita para o RH revisar (em vez de bloquear).

Testes isolados e rápidos

Testes que compartilham banco geram falsos positivos. Cada teste sobe uma app própria com SQLite em memória (`StaticPool`) criada e destruída na fixture, com CSRF e rate-limit desligados no modo TESTING.

Correção de ponto sem apagar histórico

Corrigir um ponto não pode sobrescrever o original (auditoria). Criamos a entidade `Retificacao` com fluxo de aprovação do RH (`PENDENTE/APROVADA/REJEITADA`); a correção gera novo registro do tipo `ALTERACAO/INCLUSAO`, preservando a trilha.

Em síntese, o E-Ponto é um REP-P web multi-empresa em Flask que combina arquitetura modular (factory + blueprints), persistência com ORM e migrações, validação no servidor, interface responsiva, 122 testes automatizados e conformidade legal - cobrindo os sete critérios técnicos da avaliação.